



第1讲 绪论、数值计算误差

教师：肖依永 | 刘杰 | 王洵 | 周晟瀚
可靠性与系统工程学院

内容提要

- 1.1 计算方法的对象与特点
- 1.2 误差的来源及误差的基本概念
- 1.3 机器数系
- 1.4 误差危害的防止

1.1 计算方法的对象与特点

⑩ 《计算方法》研究什么内容？有什么特点？

1 计算方法是研究数学问题的数值解及其理论的一个数学分支。

2 计算方法主要研究适合于在计算机上使用的数值计算方法及与此相关的理论

计算方法：适合于计算机运算，计算量小。

相关理论：收敛性、稳定性、误差分析等。

3 计算方法除具有数学的抽象性和严格性外，还具有应用的广泛性和实际计算的技术性，是一门与计算机密切结合的实用性很强的课程。

1.1 计算方法的对象与特点

- 计算方法解决实际问题的一般步骤：
 1. 提出实际问题
 2. 建立数学模型
 3. 提出数值问题
 4. 设计可靠、高效的算法
 5. 程序设计、上机计算
 6. 计算结果的可视化
- 数值计算已经被公认为与理论分析、实验分析并列的科学研究三大基本手段之一。

1.1 计算方法的对象与特点

本课程的学习方法

- 理解每个算法建立的数学背景。
- 熟悉相关公式，学会推导证明方法，提高理论分析和解决实际问题的能力。

重点掌握：问题、方法的思想、性能分析。

- 进行习题练习。25%
- 进行数值计算的上机实验。25%

1.2 误差的来源及误差的基本概念

1.2.1 误差的来源

一个物理量的真实值和我们计算出的值往往不相等，其差异称为误差。

➤ 模型误差

数学模型和实际问题之间的误差。建立数学模型时，对被描述的实际问题进行了抽象和简化，忽略了一些次要因素。

➤ 观测误差

对数学模型中的物理量进行观测，不可避免会带来的误差。

1.2.1 误差的来源

➤ 截断误差

数值计算中有限过程代替无限过程，从而产生的误差。也称为方法误差。如无穷级数求和，只能取前面有限项求和来近似代替，就产生了误差。

➤ 舍入误差

通过四舍五入，用有限位数进行数值计算，从而产生的计算误差。如 $1/3$ 、等，保留有限位数就会产生误差。少量舍入误差是微不足道的，但计算机上完成了千百万次运算后，舍入误差的积累可能是十分惊人的。

1.2.1 误差的来源

- 四种误差中，前两种（模型误差，观测误差）是客观存在的，后两种（截断误差，舍入误差）是计算方法和计算过程引起的。本课程只涉及后两种误差。
- 误差是不可避免的，要求绝对准确、绝对严格是办不到的，也是不必要的。
- 在计算方法中讨论的都是近似解，但应该尽量减少误差，提高精度。

1.2.2 绝对误差与绝对误差限

➤绝对误差

设 x 是准确值 x^* 的近似值，称 e 是近似值 x 的绝对误差。简称误差。

$$e = x^* - x$$

➤绝对误差限

通常准确值 x^* 未知，故绝对误差很难得到。但可以估计出 $|e|$ 不超过某个正数 ε ，称 ε 为绝对误差限。简称误差限。

$$|e| = |x^* - x| \leq \varepsilon \quad x^* = x \pm \varepsilon \quad x - \varepsilon \leq x^* \leq x + \varepsilon$$

1.2.2 绝对误差与绝对误差限

例：

用毫米刻度的直尺测量一长度为 x^* 的物体，测得其长度的近似值为 $x = 123\text{mm}$ ，由于直尺以毫米为刻度，所以其误差不超过 0.5mm ，即：

$$|x^* - 123| \leq 0.5$$

从这个不等式我们不能得出准确值 x^* ，但却知道 x^* 的范围：

$$122.5 \leq x^* \leq 123.5$$

1.2.3 相对误差与相对误差限

绝对误差的大小，在许多情况下还不能完全刻划一个近似值的准确程度。例如测量1000米和1米两个长度，若它们的绝对误差都是1厘米，显然前者的测量比较准确。由此可见一个近似值的精确程度，除了要考虑绝对误差外，还需考虑测量量本身的大小。

➤ 相对误差

设 x^* 是准确值， x 是近似值，称下式为近似值 x 的相对误差。

$$e_r = \frac{e}{x^*} = \frac{x^* - x}{x^*}$$

1.2.3 相对误差与相对误差限

实际计算中真值 x^* 通常难以求得，因此通常用下式作为相对误差。

$$\bar{e}_r = \frac{e}{x} = \frac{x^* - x}{x}$$

➤ 相对误差限

计算相对误差与计算绝对误差具有相同的困难，因此通常只考虑相对误差限 ε_r 。

$$|e_r| \leq \varepsilon_r \quad \text{或} \quad |\bar{e}_r| \leq \varepsilon_r$$

相对误差和相对误差限是无量纲的，但绝对误差和绝对误差限是有量纲的。

1.2.4 有效数字

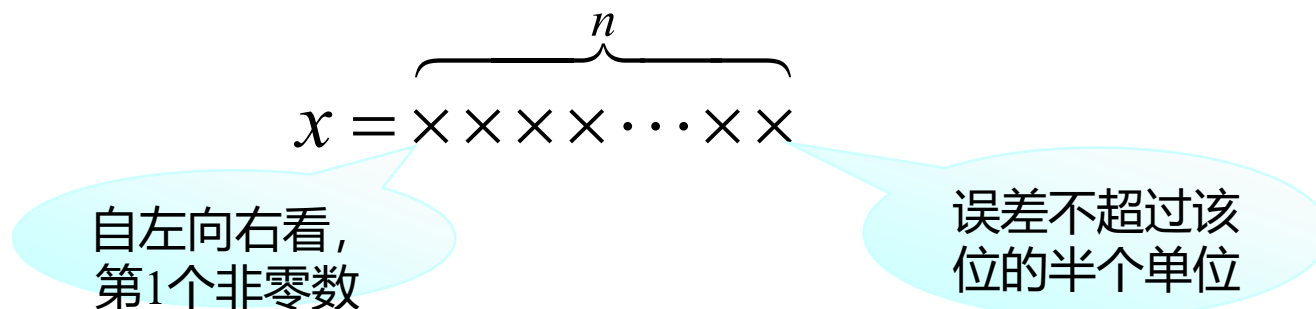
希望近似数自身就直接指示出其误差的大小。

当准确值 x^* 有很多位数时，常常按“四舍五入”的原则去得到 x^* 的近似数。例如

$$\pi = 3.1415926\cdots$$

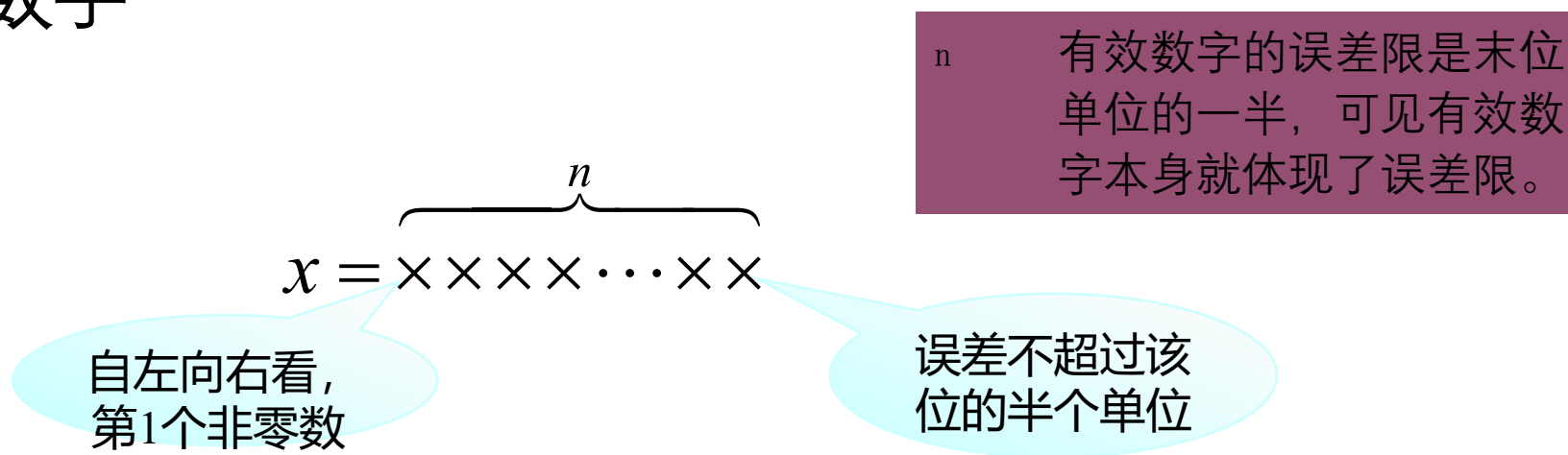
按舍入原则，若取4位小数得 $\pi = 3.1416$ ，取5位小数则有 $\pi = 3.14159$ 。

1.2.4 有效数字



有效数字（定义）： 如果近似值 x 的误差限是其某一位上的半个单位，且该位直到 x 的第1位非零数字一共有 n 位，则称近似值 x 有 n 位有效数字。

1.2.4 有效数字



- $x^* = 1.732050808$
- 取 $x_1 = 1.73$ ，则 $|e_1| = 0.002050808$ ， $|e_1| \leq 0.005$ ， $\varepsilon_1 = 0.005$ 。所以是3位有效数字。
- 取 $x_2 = 1.7321$ ，则 $|e_2| = 0.000049192$ ， $|e_2| \leq 0.00005$ ， $\varepsilon_2 = 0.00005$ 。所以是5位有效数字。
- 取 $x_3 = 1.7320$ ，则 $|e_3| = 0.000050808$ ， $|e_3| > 0.00005$ ， $|e_3| \leq 0.0005$ ， $\varepsilon_3 = 0.0005$ 。所以是4位有效数字。

1.2.4 有效数字

例：

- 0.2300有4位有效数字，而不是2位有效数字。
- 0023有2位有效数字，而不是4位有效数字。
- 12300有5位有效数字。
- 0.123×10^5 有3位有效数字，而不是5位有效数字。
- 0.12300×10^5 有5位有效数字，而不是3位有效数字。
- 数字末尾的0不可随意省去。

1.2.4 有效数字

补充：

- **精确度：**一个近似数，四舍五入到哪一位就说此近似数精确到那一位。例：
1.65精确到百分位，1.650精确到千分位。
- **近似数2.4万和24000这两个数的意义：**
 - 精确度不同，2.4万精确到千位，而24000精确到个位。
 - 有效数字不同，2.4万有两个有效数字，而24000有五个有效数字。
- **近似数的计算规则：**一般来说，进行加减运算时，中间运算过程应比要求的精确度多取一位；进行乘除运算时，中间运算过程应比要求的有效数字多取一位。

1.2.4 有效数字

例：

对下列各数写出具有5位有效数字的近似值：

236.478, 0.00234711, 9.000024, 9.000034×10^3

答案：

236.48, 0.0023471, 9.0000, 9.0000×10^3

指出下列各数有几位有效数字：

2.0004, -0.00200, -9000, 9×10^3 , 2×10^{-3}

答案：

5, 3, 4, 1, 1

1.2.4 有效数字

例：问3.142，3.141分别作为 π 的近似值各具有几位有效数字？

解 $\pi = 3.14159265\dots$ 记

$$x_1 = 3.142,$$

$$x_2 = 3.141,$$

$$x_3 = \frac{22}{7}.$$

由 $\pi - x_1 = 3.14159\dots - 3.142 = -0.00041\dots$ 知

$$\frac{1}{2} \times 10^{-4} < |\pi - x_1| \leq \frac{1}{2} \times 10^{-3},$$

因而 x_1 具有 4 位有效数字。

由 $\pi - x_2 = 3.14159\dots - 3.141 = 0.00059\dots$ 知

$$\frac{1}{2} \times 10^{-3} < |\pi - x_2| \leq \frac{1}{2} \times 10^{-2},$$

因而 x_2 具有 3 位有效数字。

1.2.5 数据误差的影响

数值运算中由于所给数据的误差必然引起函数值的误差，这种数据误差的影响比较复杂，一般采用泰勒级数展开的方法来估计。如计算 $y = f(x_1, x_2)$ 的值，设给定数据 x_1 和 x_2 是近似数，则由此计算得到的 y 也只是近似值，现在来研究 y 的绝对误差和相对误差。

x_1^*, x_2^* 是准确值 $y^* = f(x_1^*, x_2^*)$ 是函数准确值

x_1, x_2 是近似值 $y = f(x_1, x_2)$ 是函数近似值

$e(y) = y^* - y = f(x_1^*, x_2^*) - f(x_1, x_2)$ 是 y 的误差

可以利用一阶泰勒多项式进行误差的估计

1.2.5 数据误差的影响

$$f(x_1^*, x_2^*) \approx f(x_1, x_2) + \frac{\partial f(x_1, x_2)}{\partial x_1} (x_1^* - x_1) + \frac{\partial f(x_1, x_2)}{\partial x_2} (x_2^* - x_2)$$

$$e(y) = y^* - y \approx \frac{\partial f(x_1, x_2)}{\partial x_1} (x_1^* - x_1) + \frac{\partial f(x_1, x_2)}{\partial x_2} (x_2^* - x_2)$$

$$e(y) \approx \frac{\partial f(x_1, x_2)}{\partial x_1} e(x_1) + \frac{\partial f(x_1, x_2)}{\partial x_2} e(x_2)$$

进而可得

$$e_r(y) = \frac{\partial f(x_1, x_2)}{\partial x_1} \frac{x_1}{y} e_r(x_1) + \frac{\partial f(x_1, x_2)}{\partial x_2} \frac{x_2}{y} e_r(x_2)$$

1.2.5 数据误差的影响

$$e(y) \approx \frac{\partial f(x_1, x_2)}{\partial x_1} e(x_1) + \frac{\partial f(x_1, x_2)}{\partial x_2} e(x_2)$$

两数和、差、积、商的绝对误差估计：

$$y = x_1 + x_2 \quad \longrightarrow \quad e(x_1 + x_2) \approx e(x_1) + e(x_2)$$

$$y = x_1 - x_2 \quad \longrightarrow \quad e(x_1 - x_2) \approx e(x_1) - e(x_2)$$

$$y = x_1 x_2 \quad \longrightarrow \quad e(x_1 x_2) \approx x_2 e(x_1) + x_1 e(x_2)$$

$$y = \frac{x_1}{x_2} \quad \longrightarrow \quad e\left(\frac{x_1}{x_2}\right) \approx \frac{1}{x_2} e(x_1) - \frac{x_1}{x_2^2} e(x_2)$$

$x_2 \neq 0$

1.2.5 数据误差的影响

$$e_r(y) \approx \frac{\partial f(x_1, x_2)}{\partial x_1} \frac{x_1}{y} e_r(x_1) + \frac{\partial f(x_1, x_2)}{\partial x_2} \frac{x_2}{y} e_r(x_2)$$

两数和、差、积、商的相对误差估计：

$$e_r(x_1 + x_2) \approx \frac{x_1}{x_1 + x_2} e_r(x_1) + \frac{x_2}{x_1 + x_2} e_r(x_2)$$

$$e_r(x_1 - x_2) \approx \frac{x_1}{x_1 - x_2} e_r(x_1) - \frac{x_2}{x_1 - x_2} e_r(x_2)$$

$$e_r(x_1 x_2) \approx e_r(x_1) + e_r(x_2)$$

$$e_r\left(\frac{x_1}{x_2}\right) \approx e_r(x_1) - e_r(x_2) \quad x_2 \neq 0$$

1.2.5 数据误差的影响

例：已测定某物体行程 s^* 的近似值 $s=800\text{m}$ ，所需时间 t^* 的近似值 $t=35.0\text{s}$ 。若已知 $|t^* - t| \leq 0.05\text{s}$, $|s^* - s| \leq 0.5\text{m}$

试求平均速度 v 的绝对误差限和相对误差限。

$$\text{解} \quad v = \frac{s}{t} \quad e\left(\frac{x_1}{x_2}\right) \approx \frac{1}{x_2} e(x_1) - \frac{x_1}{x_2^2} e(x_2) \quad x_2 \neq 0$$

$$e(v) = e\left(\frac{s}{t}\right)$$

1.2.5 数据误差的影响

绝对误差限

$$e(v) \approx \frac{1}{t} e(s) - \frac{s}{t^2} e(t)$$

$$|e(v)| \approx \left| \frac{1}{t} e(s) - \frac{s}{t^2} e(t) \right|$$

$$\leq \frac{1}{t} |e(s)| + \frac{s}{t^2} |e(t)|$$

$$\leq \frac{1}{35} \times 0.5 + \frac{800}{35^2} \times 0.05 \approx 0.0469 \leq 0.05$$

1.2.5 数据误差的影响

相对误差限 $e_r(v) = e_r\left(\frac{s}{t}\right) \quad e_r\left(\frac{x_1}{x_2}\right) \approx e_r(x_1) - e_r(x_2)$

$$e_r(v) \approx e_r(s) - e_r(t)$$

$$\begin{aligned} |e_r(v)| &\approx |e_r(s) - e_r(t)| \\ &\leq |e_r(s)| + |e_r(t)| \\ &\leq \frac{0.5}{800} + \frac{0.05}{35} \approx 0.00205 \end{aligned}$$

1.3 机器数系

1.3.1 数的浮点表示

$$456.789 \longrightarrow 0.456789 \times 10^3$$

尾数部 定位部

一个基数为 β 的 t 位数字的浮点表示形式为：

$$x = (\pm 0.a_1 a_2 \cdots a_t) \times \beta^p$$

$\beta = 2, 8, 10, 16$
 $0 \leq a_i \leq \beta - 1$

p : 阶码, 指数
 β : 基数
 t : 字长
 $L \leq p \leq U$
计算机硬件决定

1.3.1 数的浮点表示

$$\text{尾数部 } s = \pm 0.a_1a_2 \cdots a_t = \pm \left(\frac{a_1}{\beta} + \frac{a_2}{\beta^2} + \cdots + \frac{a_t}{\beta^t} \right)$$

$$x = s \times \beta^p$$

若 $a_1 \neq 0$ 则 $\beta^{-1} \leq |s| < 1$, 称 x 为规格化浮点数

例：十进制数用二进制表示 ($\beta = 2$)

$$9.75 \text{ (十进制)} = 1001.11 \text{ (二进制)} = 1.00111 \times 2^3 \text{ (二进制)}$$

$$9.75 = \underbrace{1 \times 2^3 + 0 \times 2^2 + 0 \times 2^1 + 1 \times 2^0}_{\text{整数部}} + \underbrace{(1 \times 2^{-1} + 1 \times 2^{-2})}_{\text{小数部}}$$

1.3.1 数的浮点表示

将十进制数转化为 n 位二进制数的方法：

(1) 将十进制数 x 分为整数部分 a 和小数部分 b

对整数部分：

(2) 令 $k=1$, $a_k \leftarrow a$

(3) 对整数部分 a_k **除以2**，得到商 a_{k+1} ，将余数作为其整数二进制的第 k 位（从小数点往左）

(4) 重复第 (3) 部，**直到** $a_{k+1}=0$ **或者** $k=n$

对小数部分：

(5) 令 $k'=1$, $b_{k'} \leftarrow b$

(6) 对小数部分 $b_{k'}$ **乘以2**，得到积 $b_{k'+1}$ ，将整数作为其小数二进制的第 k' 位（从小数点往右）

(7) 去掉 $b_{k'+1}$ 的整数部分，重复第 (6) 部，**直到** $b_{k'+1}=0$ **或** $k'=n-k$ 。

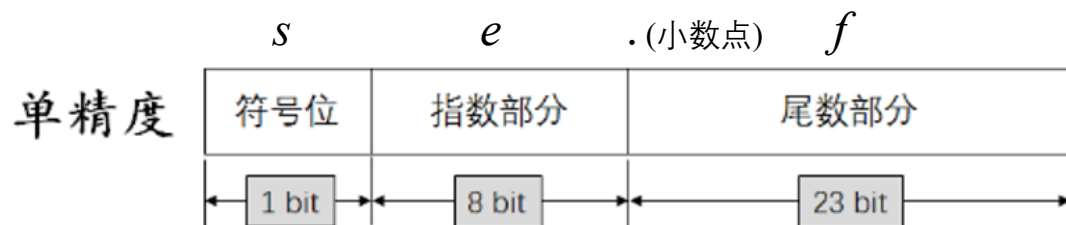
例子：10.75（十进制）= 1010.11（二进制）

整数部分：10÷2=5余**0**，5÷2=2余**1**，2÷2=1余**0**，1÷2=0余**1**，返序得到1010

小数部分：0.75×2=1.5取整**1**，0.5×2=1.0取整**1**，得到11

1.3.1 数的浮点表示

计算机中的单精度浮点数存储（32位）：



其中： s 表示符号，0为正，1为负；

e 表示小数点右移的位置，即尾数部分从左到右第 e 位之后

f 由整数和小数组成的数值（小数点位置由 e 来确定）

例：十进制数 **9.75** 的二进制存储格式为 0 10000010 (1)001.11000000000000000000

10000010 - 01111111 = 3 (小数点右移3位)

十进制数 **925.7525** 的二进制存储格式为 0 10001000 (1)110011101.11000000101001

10001000 - 01111111 = 9 (小数点右移9位)

十进制数 **-10.75** 的二进制存储格式为 1 10000010 (1)010.11000000000000000000

10001000 - 01111111 = 3 (小数点右移3位)

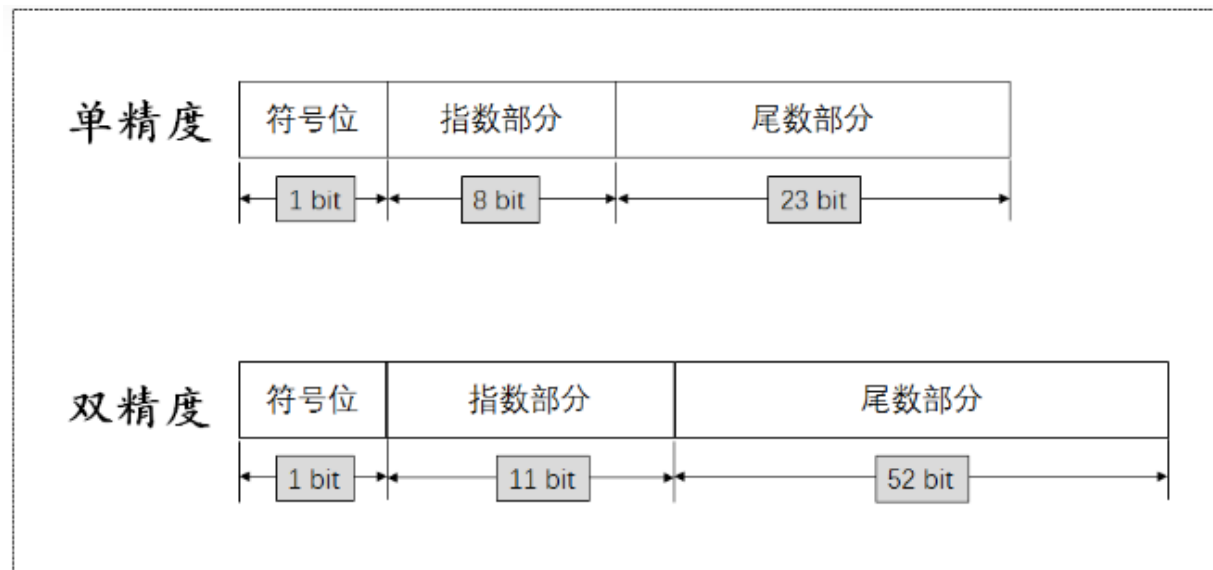
十进制数 **-0.0125** 的二进制存储格式为 1 01111000 (1)10011001100110011001101

01111000 - 01111111 = -7 (小数点左移7位)

.0000000(1)1001100110011001

1.3.1 数的浮点表示

单精度（32位）、双精度（64位）存储:



单精度: 正最大值=(0 1111 1111 [1]111 1111 1111 1111 1111 1111)₂

负最大值=(1 1111 1111 [1]111 1111 1111 1111 1111 1111)₂

正最小数=(0 0000 0000 [1]000 0000 0000 0000 0000 0000)₂

1.3.2 机器数系

- 前述数的浮点表示方法是当今计算机采用的表示方法。
- 把计算机中浮点数所组成的集合加上“机器零”，记为F，则F被以下4个参数所描述：

β	基数
t	字长
$[L, U]$	阶码范围

- 集合F称为机器数系。
- 机器数系F是一个离散的分布不均匀的有限集。
- 若计算的结果超出了集合F的范围，则称为溢出。

1.4 误差危害的防止

1.4.1 使用数值稳定的计算公式

什么是数值稳定的计算公式？先看一个例子

例2 建立积分 $I_n = \int_0^1 \frac{x^n}{x+5} dx$, $n = 0, 1, 2, \dots, 20$ 的递推关系, 并研究它的误差传递。

解: 由于 $I_n + 5I_{n-1} = \int_0^1 \frac{x^n + 5x^{n-1}}{x+5} dx = \int_0^1 x^{n-1} dx = \frac{1}{n}$

$$I_0 = \int_0^1 \frac{1}{x+5} dx = \ln 6 - \ln 5 = \ln(1.2)$$

1.4.1 使用数值稳定的计算公式

递推公式

$$\begin{cases} I_0 = \ln(1.2) \\ I_n = \frac{1}{n} - 5I_{n-1}, \quad (n = 1, 2, \dots, 20) \end{cases}$$

在计算 I_0 时有舍入误差 e_0 , 设 I_0 的近似值为 $\bar{I}_0$

$$\begin{cases} \bar{I}_n = \frac{1}{n} - 5\bar{I}_{n-1} & n = 1, 2, \dots, 20 \\ \bar{I}_0 = I_0 - e_0 \end{cases}$$

1.4.1 使用数值稳定的计算公式

相减得到 $I_n - \bar{I}_n = (-5)(I_{n-1} - \bar{I}_{n-1}) \quad n = 1, 2, \dots, 20$

进而 $I_n - \bar{I}_n = (-5)^n e_0 \quad n = 1, 2, \dots, 20$

由此看出，误差 e_0 对第 n 步的影响是误差扩大 5^n 倍，当 n 较大时，误差将淹没真值。

因此，用 $\bar{I}_n$ 近似 I_n 是不正确的，也就是说用这种递推公式求解问题是不正确的。

1.4.1 使用数值稳定的计算公式

但是如果我们由 $I_n = \frac{1}{n} - 5I_{n-1}$

得到 $I_{n-1} = \frac{1}{5n} - \frac{1}{5}I_n \quad n = 20, 19, \dots, 1$

则可以先求出 I_{20} ，再由上式依次得到其它值 $I_{19}, I_{18}, \dots, I_0$

使用广义积分中值定理，可得 $\frac{1}{6 \times 21} < I_{20} < \frac{1}{5 \times 21}$

取 $I_{20} \approx \frac{\frac{1}{126} + \frac{1}{105}}{2} \approx 0.0087301587$

1.4.1 使用数值稳定的计算公式

则问题的第二种递推公式可表示为

$$\begin{cases} \bar{I}_{n-1} = \frac{1}{5n} - \frac{1}{5} \bar{I}_n & n = 20, 19, \dots, 1 \\ \bar{I}_{20} = 0.0087301587 \end{cases}$$

研究 I_{20} 的舍入误差 $e_{20} = I_{20} - \bar{I}_{20}$ 的传递情况

$$I_{n-1} - \bar{I}_{n-1} = \left(-\frac{1}{5}\right)(I_n - \bar{I}_n) \quad n = 20, 19, \dots, 1$$

$$\text{进而} \quad I_0 - \bar{I}_0 = \left(-\frac{1}{5}\right)^{20} e_{20}$$

所以第二种方法的误差传递是逐步缩小的。

1.4.1 使用数值稳定的计算公式

- 结论：
- 把计算过程中舍入误差对计算结果影响不大的算法称为数值稳定的算法，影响严重的算法称为数值不稳定的算法。
- 如果第 $n+1$ 步的误差 e_{n+1} 与第 n 步的误差 e_n 满足

$$\left| \frac{e_{n+1}}{e_n} \right| \leq 1$$

- 则称此计算公式是绝对稳定的，否则称为不是绝对稳定的。

1.4.2 尽量避免两相近数相减

根据两数差的相对误差估计式：

$$e_r(x_1 - x_2) \approx \frac{x_1}{x_1 - x_2} e_r(x_1) - \frac{x_2}{x_1 - x_2} e_r(x_2)$$

可以看出，当两个相近的数 x_1 和 x_2 相减时， $|e_r(x_1 - x_2)|$ 可能比 $|e_r(x_1)| + |e_r(x_2)|$ 大得多，从而可能严重损失有效数字。

1.4.2 尽量避免两相近数相减

例1: $a_1 = 0.12345$, $a_2 = 0.12346$, 各有5位有效数字。而 $a_2 - a_1 = 0.00001$, 只剩下1位有效数字。

例2: 计算 $\sqrt{2001} - \sqrt{1999}$

解: 记 $x_1^* = \sqrt{2001}$, $x_2^* = \sqrt{1999}$, 它们的六位有效数字分别为 $x_1 = 44.7325$, $x_2 = 44.7102$

方法一:

$$x_1^* - x_2^* \approx x_1 - x_2 = 44.7325 - 44.7102 = 0.0223$$

1.4.2 尽量避免两相近数相减

方法二：

$$x_1^* - x_2^* = \frac{2}{x_1^* + x_2^*} \approx \frac{2}{x_1 + x_2} = \frac{2}{44.7325 + 44.7102} \approx 0.0223607$$

分析两种方法的精度：

$$\begin{aligned} \text{(一)} \quad |e(x_1 - x_2)| &\approx |e(x_1) - e(x_2)| \leq |e(x_1)| + |e(x_2)| \\ &\leq \frac{1}{2} \times 10^{-4} + \frac{1}{2} \times 10^{-4} = 10^{-4} < \frac{1}{2} \times 10^{-3} \end{aligned}$$

所以第一种方法只能断定其结果至少具有2位有效数字。

1.4.2 尽量避免两相近数相减

$$\begin{aligned} \text{(二)} \quad & \left| e\left(\frac{2}{x_1 + x_2}\right) \right| \approx \left| -\frac{2}{(x_1 + x_2)^2} e(x_1 + x_2) \right| \\ & \approx \left| -\frac{2}{(x_1 + x_2)^2} [e(x_1) + e(x_2)] \right| \\ & \leq \frac{2}{(x_1 + x_2)^2} [|e(x_1)| + |e(x_2)|] \end{aligned}$$

1. 4. 2 尽量避免两相近数相减

$$\leq \frac{2}{(44.7325 + 44.7102)^2} \left(\frac{1}{2} \times 10^{-4} + \frac{1}{2} \times 10^{-4} \right)$$

$$\approx 0.25 \times 10^{-7} < \frac{1}{2} \times 10^{-7}$$

所以按第二种方法结果至少具有6位有效数字。

1.4.3 尽量避免用绝对值很大的数作乘数

因为 $e(x_1x_2) \approx x_2e(x_1) + x_1e(x_2)$

$$e(y) \approx \frac{\partial f(x_1, x_2)}{\partial x_1} e(x_1) + \frac{\partial f(x_1, x_2)}{\partial x_2} e(x_2)$$

当乘数 x_1 或 x_2 的绝对值很大时, $|e(x_1x_2)|$ 可能很大。

因为 $e\left(\frac{x_1}{x_2}\right) \approx \frac{1}{x_2}e(x_1) - \frac{x_1}{x_2^2}e(x_2)$

当 x_2 接近0时, $\left|e\left(\frac{x_1}{x_2}\right)\right|$ 也可能很大。

结论: 应尽量避免用绝对值很大的数作乘数, 或用接近0的数作除数, 否则会严重影响结果的精度, 减少有效数字位数。

1.4.4 防止大数“吃掉”小数

数值运算中参加运算的数有时数量级相差很大，而计算机位数有限，又要做对阶处理，如不注意运算次序，就可能出现大数“吃掉”小数的现象，影响结果的可靠性。

例 在一台4位十进制计算机上计算

$$S = A + \delta, \text{ 其中 } A = 10000, \delta = 1.$$

$$\text{解 } fl(A) = 0.1000 \times 10^5, fl(\delta) = 0.1000 \times 10^1$$

$$\begin{aligned} fl(S) &= fl(A) + fl(\delta) = 0.1000 \times 10^5 + 0.1000 \times 10^1 \\ &= 0.1000 \times 10^5 + 0.0000 \times 10^5 \end{aligned}$$

1.4.4 防止大数“吃掉”小数

$$= (0.1000 + 0.0000) \times 10^5$$

$$= 0.1000 \times 10^5$$

计算机作加法运算时，首先要对加数作对阶处理，由于 δ 与A相比是一个小量，同时计算机字长有限，所以 δ 被作为机器零处理了。与A相加实际上是被A“吃掉”了。

结论：多个数相加，应按绝对值从小到大的顺序依次相加。

1.4.5 注意简化计算步骤，减少运算次数

同样一个计算问题，如果能减少运算次数，不但可以节省计算机的计算时间，还能减少舍入误差，这是数值计算必须遵守的原则，也是计算方法要研究的重要内容。

例：计算 x^{31} 的值。

1) 逐个相乘 30次乘法

$$2) \ x_2 = xx \quad x_4 = x_2x_2 \quad x_8 = x_4x_4 \quad x_{16} = x_8x_8$$

$$x^{31} = xx^2x^4x^8x^{16} = xx_2x_4x_8x_{16} \quad 8\text{次乘法}$$

1.4.5 注意简化计算步骤，减少运算次数

计算多项式的值。 $f(x) = a_0x^n + a_1x^{n-1} + \cdots + a_{n-1}x + a_n$

若直接计算 a_ix^{n-i} 再逐项相加，一共需做

运算量：加法： n 次

乘法： $n + (n-1) + \cdots + 2 + 1 = \frac{n(n+1)}{2}$ 次

采用秦九韶法：

$$f(x) = (a_0x^{n-1} + a_1x^{n-2} + \cdots + a_{n-1})x + a_n$$

$$f(x) = [(a_0x^{n-2} + a_1x^{n-3} + \cdots + a_{n-1})]x + a_n$$

$$f(x) = \{ \cdots \cdots \cdots \}$$

1.4.5 注意简化计算步骤，减少运算次数

$$f(x) = \{\cdots \quad \quad \quad \cdots \quad \quad \quad \}$$

$$b_0 = a_0$$

$$b_1 = b_0x + a_1$$

$$b_2 = b_1x + a_2$$

$$\vdots$$

$$b_k = b_{k-1}x + a_k$$

$$\vdots$$

$$b_n = b_{n-1}x + a_n = f(x)$$

$$\text{得递推公式: } \begin{cases} b_k = b_{k-1}x + a_k \\ b_0 = a_0 \end{cases} \quad k = 1, 2, \cdots$$

从而 $f(x) = b_n$ 秦九韶法运算量: n 次加法和 n 次乘法。

1.4.5 注意简化计算步骤，减少运算次数

$$f(x) = \{ \cdots \quad \quad \quad \cdots \quad \quad \quad \}$$

手算：

$$\begin{array}{cccccc}
 & a_0 & a_1 & a_2 & \cdots & a_{n-1} & a_n \\
 x = x_0 & & b_0 x_0 & b_1 x_0 & \cdots & b_{n-2} x_0 & b_{n-1} x_0 \\
 \hline
 & b_0 & b_1 & b_2 & \cdots & b_{n-1} & b_n = f(x_0)
 \end{array}$$

1.4.5 注意简化计算步骤，减少运算次数

秦九韶法

⑩ 例：求 $f(x)=2+x-x^2+3x^4$ 在 $x=2$ 的值。

⑩ 解：

$$\begin{array}{rcccccc} & & 3 & 0 & -1 & 1 & 2 \\ x=2 & & 6 & 12 & 22 & 46 & \\ \hline & & 3 & 6 & 11 & 23 & 48 = f(2) \end{array}$$

⑩ 得 $f(x)=48$

本章小结

- 计算方法的对象与特点
- 误差的来源及误差的基本概念
- 机器数系
- 误差危害的防止
- （使用数值稳定的计算公式；秦九韶法）

本章小结

⑩ 误差的基本概念

⑩ 绝对误差

$$e = x^* - x$$

⑩ 绝对误差限

$$|e| = |x^* - x| \leq \varepsilon$$

⑩ 相对误差

$$e_r = \frac{e}{x^*} = \frac{x^* - x}{x^*}$$

⑩ 相对误差限

$$|e_r| \leq \varepsilon_r$$

本章小结

- 有效数字
- 有效数字（定义）：如果近似值 x 的误差限是其某一位上的半个单位，且该位直到 x 的第1位非零数字一共有 n 位，则称近似值 x 有 n 位有效数字。
- 例：
- $x^* = 1.732050808$
- 取 $x_1 = 1.73$ ，则 $|e_1| = 0.002050808$ ， $|e_1| \leq 0.005$ ， $\varepsilon_1 = 0.005 = 0.5 \times 10^{-2}$ 。所以是3位有效数字。

本章小结

- 误差危害的防止
- 1 使用数值稳定的计算公式
- 2 尽量避免两相近数相减
- 3 尽量避免用绝对值很大的数作乘数
- 4 防止大数“吃掉”小数
- 5 注意简化计算步骤，减少运算次数
- （秦九韶法）

作业

1. 教材P15页，第9、10、12题。



谢谢各位同学！

计算实验

- 1 熟悉计算环境
- 2 根据单、双精度浮点数格式原理，验证C语言单/双精度变量的有效取值范围
- 3 利用秦九韶法的计算效率比较实验

(1) 熟悉计算环境

- 1 计算环境Linux，标准c语言cc编译器
- 2 SSH远程登录服务器，登录用户名、密码
- 3 编写hello world程序，试编译和运行。

(2) 验证C语言单双精度变量的有效取值范围

C语言代码参考例子：

```
int main(int argc, char* argv[])
{
    printf("Hello World!\n");
    //双精度浮点数
    double a;
    a = 1.7976931348623157e+308;           //双精度最大数
    printf("a=%30.29e\n", a);
    a=a+0.000000000000000001e+308;       //增加最小有效数，均导致出错
    printf("a=%30.29e\n", a);

    //单精度浮点数
    float b;
    b = 3.4028234e+38;                    //单精度最大数
    printf("b=%30.29e\n", b);

    getchar();
    return 0;
}
```

(3) 利用秦九韶法的计算效率比较实验

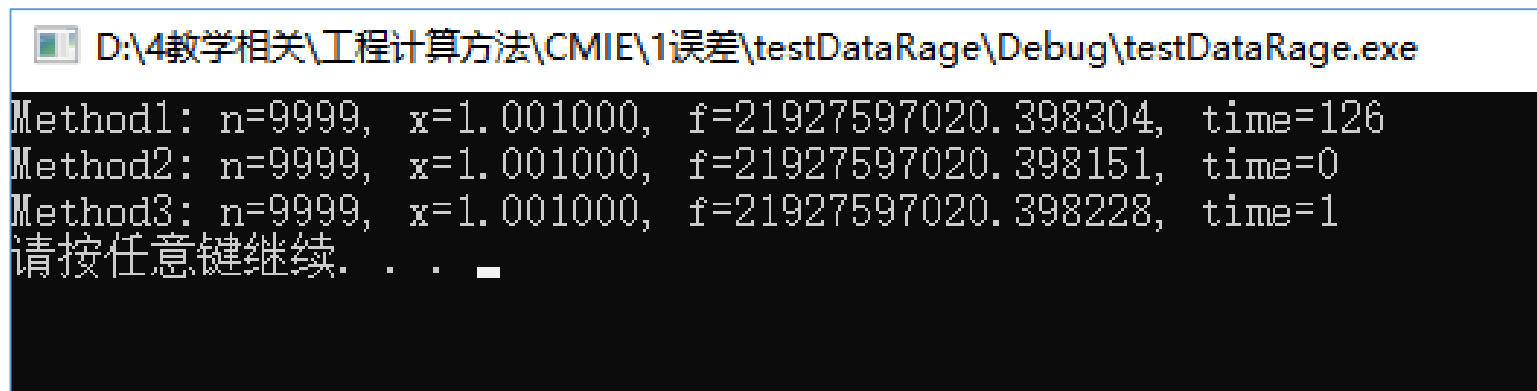
$$f(x) = a_0x^n + a_1x^{n-1} + \cdots + a_n$$

令 $x = 1.001$, $n = 9999$, $a_n = 1 + n$

比较：直接累计计算法和秦九韶法计算法，仅采用加法和乘法（不用指数函数）

$$f(x) = \{ \cdots \}$$

观察：（1）计算时间差异；（2）计算结果差异；（3）与采用指数函数计算结果比较



```
D:\4教学相关\工程计算方法\CMIE\1误差\testDataRage\Debug\testDataRage.exe
Method1: n=9999, x=1.001000, f=21927597020.398304, time=126
Method2: n=9999, x=1.001000, f=21927597020.398151, time=0
Method3: n=9999, x=1.001000, f=21927597020.398228, time=1
请按任意键继续. . .
```